

ECE0202 Lecture 15 中文逐页讲义

Interrupts 3: EXTI 外部中断与 Stepper Motor 步进电机

根据课堂 PPT 整理；面向完全初学者；字体较小以容纳详细解释。

这节课主线：

前半部分讲 **EXTI** (External Interrupt, 外部中断)：如何让 STM32L4 的一个 GPIO 引脚，例如 PA3 joystick up button，在电平变化时触发中断，让 CPU 自动跳到 EXTI3_IRQHandler。后半部分讲 **Stepper Motor** (步进电机)：为什么它可以精准转动，如何用 GPIO 输出四相控制序列让它转动。

Contents

1	Slides 1-2: GPIO 总入口	2
2	Slides 3-6: Joystick 与 Push Button 是中断事件来源	3
3	Slide 7: EXTI Sources - PA3 为什么对应 EXTI3	4
4	Slides 8-11: SYSCFG_EXTICR1-4 怎么选 EXTI 输入源	5
5	Slide 12: Interrupt Vector Table 与 EXTI0-EXTI4	6
6	Slides 13-24: EXTI Controller 寄存器总览	7
7	Slide 25: 实验目标 - 按 Up Button 点亮 LED	8
8	Slide 26: Enable GPIO Clock - 先给 GPIOA 通电/供时钟	9
9	Slide 27: 把 PA3 设置为 Input Mode	10
10	Slide 28: 设置 PA3 Internal Pull-down	11
11	Slide 29: 配置 NVIC - 让 CPU 允许响应 EXTI3	12
12	Slide 30: 连接 PA3 到 EXTI3 - SYSCFG 配置	13
13	Slides 31-32: Rising Trigger Selection - 让上升沿触发	14
14	Slides 33-35: Interrupt Mask Register 和主程序死循环	15
15	Slide 36: EXTI3_IRQHandler - 中断服务程序	16
16	Slides 37-38: 普通电机 vs 步进电机	17
17	Slide 39: 28BYJ-48 与 ULN2803 驱动连接	18
18	Slides 40-41: 步进电机内部结构与磁极激活	19
19	Slide 42: Full-Stepping 全步进	20
20	Slides 43-44: Half-Stepping 半步进	21
21	Slides 45-46: 一圈需要多少步	22
22	Lecture 15 全链路总结	23

Slides 1-2: GPIO 总入口

这两页在说什么

Lecture 15 从 **GPIO** (General Purpose Input/Output, 通用输入输出) 开始。STM32L4 有多个 GPIO port: A、B、C、D、E、F、G、H。每个 port 最多 16 个 pin, 所以 PA3 的含义就是: **Port A** 的第 **3** 号引脚。

最基础模型:

- **GPIOA**: A 端口, 一组最多 16 个引脚。
- **PA3**: A 端口的第 3 个引脚。
- **GPIOB**: B 端口。
- **PB2**: B 端口的第 2 个引脚。

为什么这里又讲 GPIO?

因为外部中断本质上经常来自 GPIO 引脚。比如按键、摇杆、传感器输出脚, 它们在物理世界中产生电压变化。CPU 不会直接“感觉到按键”, 而是通过 GPIO 输入电平读到变化。EXTI 再把这种变化变成中断请求。

关键词

- **GPIO Port** (GPIO 端口): 一组 GPIO 引脚的集合, 例如 GPIOA。
- **Pin** (引脚): 芯片实际连接外部电路的脚, 例如 PA3。
- **Input** (输入模式): CPU 读取外部电平。
- **Output** (输出模式): CPU 控制引脚输出高低电平。

易错点: GPIOA 不是一个引脚, 而是一整个端口; PA3 才是具体的一个引脚。

Slides 3-6: Joystick 与 Push Button 是中断事件来源

这几页在说什么

PPT 展示 STM32L4 Discovery Kit 上的 **Joystick** (摇杆/方向键)。它有 Up、Down、Left、Right、Center 几个方向。每一个方向本质上都可以看成一个 **Push Button** (按键)。按下时，引脚电压发生变化；松开时，引脚电压恢复。

物理层直白解释

以 Up button 为例，它连接到 PA3。图中 PA3 一端接到按钮结构，按钮另一端接到 +3V。按下按钮时，PA3 会被拉到高电平；如果没有按下，PA3 的状态要靠上拉/下拉电阻确定。

最直白理解：

- 没按按钮：PA3 可能是低电平，取决于下拉电阻。
- 按下按钮：PA3 被连接到 +3V，所以变成高电平。
- 低电平到高电平的变化叫 **rising edge** (上升沿)。
- EXTI 可以被配置成看到上升沿就产生中断。

为什么不能只用 polling?

你当然可以在 while(1) 里不断读 PA3：

```
while (1) {
    if (GPIOA->IDR & (1UL << 3)) {
        // button pressed
    }
}
```

但这样 CPU 一直在问“按了吗？按了吗？按了吗？”。这叫 **polling** (轮询)，效率低。中断方式是：CPU 正常干别的，按键真正发生时再通知 CPU。

关键区别

方式	英文	本质
轮询	Polling	CPU 主动反复检查事件是否发生
中断	Interrupt	事件发生后硬件主动通知 CPU

易错点：按钮不是“直接调用函数”。按钮只造成电压变化；GPIO/EXTI/NVIC 这一整套硬件链路才让 CPU 进入 ISR。

Slide 7: EXTI Sources - PA3 为什么对应 EXTI3

核心问题

这一页最关键：为什么 **PA3**、**PB3**、**PC3**、**PD3** 等都可能连到 **EXTI3**？

EXTI line 的编号对应的是 pin number，不是 port number。所以：

- PA0、PB0、PC0、... 对应 EXTI0。
- PA1、PB1、PC1、... 对应 EXTI1。
- PA3、PB3、PC3、... 对应 EXTI3。

但同一个 EXTI line 不能同时接多个 port 的同号 pin。你必须通过 **SYSCFG_EXTICR**（系统配置外部中断配置寄存器）选择到底是哪一个 port。

图中的路线

以 PA3 为例：

1. 外部按键连接到 PA3。
2. SYSCFG_EXTICR 选择 source selection = 000，即选择 Port A。
3. 因为 pin number 是 3，所以进入 EXTI3。
4. EXTI3 向 **NVIC**（Nested Vectored Interrupt Controller，嵌套向量中断控制器）发中断请求。
5. NVIC 让 Cortex-M4 跳到 EXTI3_IRQHandler。

一句话：PA3 的数字 3 决定它走 EXTI3；字母 A 需要由 SYSCFG_EXTICR 选择。

关键英文词汇

- **EXTI**（External Interrupt/Event Controller，外部中断/事件控制器）
- **Source selection**（源选择）：决定 EXTIx 来自哪个 GPIO port。
- **NVIC**（Nested-Vectored Interrupt Controller，嵌套向量中断控制器）
- **IRQHandler**（Interrupt Request Handler，中断请求处理函数）

易错点：EXTI3 不是“PA3 专属”。PA3、PB3、PC3 等都可以接 EXTI3，但同一时刻必须通过 SYSCFG 选一个。

Slides 8-11: SYSCFG_EXTICR1-4 如何选择 EXTI 输入源

这组页在说什么

这组页展示 SYSCFG_EXTICR1 到 SYSCFG_EXTICR4。它们的作用是：把某个 **GPIO port** 的某个 **pin** 连接到对应的 **EXTI line**。

为什么有 4 个 EXTICR?

STM32 有 EXTI0、EXTI1、...、EXTI15 这些 GPIO 外部中断线。每条线需要选择来自 A/B/C/... 哪个 port。16 条线如果全部塞进一个寄存器太拥挤，所以拆成 4 个寄存器：

寄存器	管理的 EXTI line	例子
SYSCFG_EXTICR1	EXTI0-EXTI3	PA3 对应 EXTI3, 在这里配置
SYSCFG_EXTICR2	EXTI4-EXTI7	PA5 对应 EXTI5, 在这里配置
SYSCFG_EXTICR3	EXTI8-EXTI11	PC13 对应 EXTI13, 不在这里
SYSCFG_EXTICR4	EXTI12-EXTI15	PC13 对应 EXTI13, 在这里配置

PA3 的配置逻辑

PA3 对应 EXTI3。EXTI3 在 SYSCFG_EXTICR1 里配置。source selection 为：

- 000: 选择 PA3。
- 001: 选择 PB3。
- 010: 选择 PC3。
- 011: 选择 PD3。
- 以此类推。

```
LDR r0, =SYSCFG_BASE
LDR r1, [r0, #SYSCFG_EXTICR0]
BIC r1, #SYSCFG_EXTICR1_EXTI3 ; clear EXTI3 source bits
ORR r1, #SYSCFG_EXTICR1_EXTI3_PA ; select PA3
STR r1, [r0, #SYSCFG_EXTICR0]
```

逐行解释

- LDR r0, =SYSCFG_BASE: 把 SYSCFG 模块的基地址放入 r0。
- LDR r1, [r0, ##SYSCFG_EXTICR0]: 读出 EXTICR1 当前值。
- BIC: Bit Clear, 先把 EXTI3 对应位清零, 避免旧配置残留。
- ORR: Bitwise OR, 把 EXTI3 source 设置成 PA。
- STR: 把修改后的配置写回寄存器。

易错点: 配置 SYSCFG_EXTICR 只是告诉 EXTI “EXTI3 来自 PA3”。它还没有配置上升沿/下降沿, 也没有解除 mask, 也没有启用 NVIC。

Slide 12: Interrupt Vector Table 与 EXTI0-EXTI4

这页在说什么

Interrupt Vector Table（中断向量表）保存每个中断对应 ISR 的地址。对外部中断来说，EXTI0、EXTI1、EXTI2、EXTI3、EXTI4 各自有独立的中断入口。

向量表的作用：

中断发生后，CPU 不会靠函数名猜入口地址。NVIC 根据中断号查 vector table，找到对应 ISR 地址，再把 PC 改成这个地址，CPU 才开始执行 ISR。

为什么只说 first 5 dedicated external interrupts?

EXTI0 到 EXTI4 是独立入口：

- EXTI0 对应 EXTI0_IRQHandler
- EXTI1 对应 EXTI1_IRQHandler
- EXTI2 对应 EXTI2_IRQHandler
- EXTI3 对应 EXTI3_IRQHandler
- EXTI4 对应 EXTI4_IRQHandler

后面的 EXTI line 往往会合并入口，例如 EXTI9_5、EXTI15_10。合并入口意味着 ISR 里还要检查到底是哪一条 line 触发。

PA3 的路线

```
PA3 voltage rising edge
-> EXTI3 pending flag set
-> NVIC sees EXTI3 interrupt request
-> vector table gives address of EXTI3_IRQHandler
-> CPU executes EXTI3_IRQHandler
```

易错点：向量表保存的是函数入口地址，不是函数代码本身，也不是中断编号本身。

Slides 13-24: EXTI Controller 寄存器总览

这组页的核心

EXTI controller 不是只靠一个开关。它至少涉及以下寄存器：

寄存器	英文全称	直白作用
RTSR	Rising Trigger Selection Register	选择上升沿是否触发
FTSR	Falling Trigger Selection Register	选择下降沿是否触发
IMR	Interrupt Mask Register	是否允许中断请求通过
EMR	Event Mask Register	是否允许事件通过，通常不等于中断
PR	Pending Register	记录某条 EXTI 是否已经触发
SWIER	Software Interrupt/Event Register	软件手动制造一个 EXTI 触发

一条外部中断的硬件链路

以 PA3 上升沿触发 EXTI3 为例：

1. PA3 输入电压从 0 变成 1。
2. RTSR1 的 RT3 位如果为 1，就认可这个上升沿。
3. EXTI3 的 PR1 pending flag 被置 1，表示“中断请求发生了”。
4. IMR1 的 IM3 位如果为 1，说明没有被 mask，中断请求可以进入 NVIC。
5. NVIC 如果也 enable 了 EXTI3，就让 CPU 进入 EXTI3_IRQHandler。

必须分清两层 enable：

- EXTI 里的 IMR：允许 EXTI line 把中断请求送出去。
- NVIC 里的 enable：允许 CPU 响应这个中断请求。

两层缺一不可。只开一个，程序不会按预期进入 ISR。

RTSR 和 FTSR

- **Rising edge**（上升沿）：电平从 0 变 1。
- **Falling edge**（下降沿）：电平从 1 变 0。
- 如果按钮按下导致 PA3 从低变高，就用 RTSR。
- 如果按钮按下导致 PA3 从高变低，就用 FTSR。
- 两个都开，则上升沿和下降沿都触发。

PR 的特殊清除方式

PR 是 **Pending Register**（挂起寄存器）。它记录某个 EXTI line 是否触发过。很多初学者会以为清除 pending flag 要写 0，实际这类寄存器常常是：**write 1 to clear**，写 1 清除。

重点：如果 ISR 里不清除 PR1 的 PIF3，CPU 可能会反复进入同一个中断。你以为程序卡死，实际是 pending flag 没清掉。

Slide 25: 实验目标 - 按 Up Button 点亮 LED

软件目标

PPT 写的是: **If the up button is pressed, turn on LED.** 即按下 joystick 的 up button 时点亮 LED。这个例子用 PA3 作为中断输入源, PA3 对应 EXTI3。

为什么需要 pull-down?

PPT 说明 PA3 externally neither pulled up nor pulled down, 即外部没有上拉/下拉。没有上拉/下拉时, 引脚在未按下时可能是 **floating** (浮空) 状态。浮空状态下读到 0 或 1 都可能, 容易产生假中断。

本例策略:

- 未按下: 内部下拉电阻把 PA3 拉到 0。
- 按下: 按钮把 PA3 接到 +3V, PA3 变成 1。
- 所以按下瞬间是 0 -> 1, 即上升沿。
- 因此配置 RTSR1, 让 rising edge 触发中断。

完整配置路径

```
GPIOA clock enable
-> PA3 input mode
-> PA3 internal pull-down
-> SYSCFG selects PA3 as EXTI3 source
-> EXTI3 rising trigger enabled
-> EXTI3 unmasked in IMR1
-> NVIC enables EXTI3 interrupt
-> ISR handles interrupt and clears pending flag
```


Slide 26: Enable GPIO Clock - 先给 GPIOA 通电/供时钟

为什么第一步是 enable clock?

STM32 的外设默认很多是关闭时钟的。原因很现实：省电。GPIOA 如果没有时钟，写 GPIOA 的寄存器可能没有效果。因此配置 PA3 前，要先打开 GPIOA 的时钟。

```
LDR r0, =RCC_BASE
LDR r1, [r0, #RCC_AHB2ENR]
ORR r1, r1, #RCC_AHB2ENR_GPIOAEN
STR r1, [r0, #RCC_AHB2ENR]
```

逐行解释

- RCC: Reset and Clock Control, 复位与时钟控制模块。
- AHB2ENR: AHB2 peripheral clock enable register, AHB2 外设时钟使能寄存器。
- GPIOAEN: GPIOA Enable, 打开 GPIOA 时钟的 bit。
- ORR: 用 OR 把某一位置 1, 不破坏其他位。

C 语言等价写法

```
RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;
```

易错点：enable GPIO clock 不是把 PA3 设置为输入，也不是开中断。它只是让 GPIOA 模块开始工作。

Slide 27: 把 PA3 设置为 Input Mode

MODER 是什么

GPIOx_MODER 是 GPIO mode register。每个 pin 用 2 个 bits 表示模式：

Mode bits	含义
00	Input, 输入模式
01	Output, 输出模式
10	AF, Alternate Function, 复用功能
11	Analog, 模拟模式, 复位默认常见状态

PA3 是 pin 3, 每个 pin 两位, 所以 PA3 的 MODER 位是 bit 7:6。二进制掩码就是 0xC0, 因为:

$$0xC0 = 1100\ 0000_2$$

对应 bit 7 和 bit 6。

```
LDR r0, =GPIOA_BASE
LDR r1, [r0, #GPIO_MODER]
BIC r1, r1, #0xC0
STR r1, [r0, #GPIO_MODER]
```

为什么用 BIC?

BIC r1, r1, ##0xC0 的意思是清除 bit7:bit6。清除后它们变成 00, 即 input mode。

关键: BIC = Bit Clear, 常用于把某些配置位清零。PA3 要输入模式, 所以 MODER3[1:0] = 00。

易错点: 不要把 0xC0 当成引脚编号。它只是用于清除 PA3 模式位的 bit mask。

Slide 28: 设置 PA3 Internal Pull-down

为什么需要下拉

如果 PA3 外部没有固定电阻, 按钮没按下时 PA3 可能浮空。浮空不是高也不是低, 是不稳定状态。为了让没按下时稳定为 0, 需要打开内部 **pull-down resistor** (下拉电阻)。

PUPDR 是什么

GPIOx_PUPDR 是 pull-up/pull-down register。每个 pin 两位:

PUPDR bits	含义
00	No pull-up, no pull-down, 无上拉下拉
01	Pull-up, 上拉
10	Pull-down, 下拉
11	Reserved, 保留

PA3 对应 bit7:6。要设置为 10, 所以:

- 先用 BIC r1, r1, ##0xC0 清零 bit7:6。
- 再用 ORR r1, r1, ##0x80 设置 bit7 = 1, bit6 = 0。

```
LDR r1, [r0, #GPIO_PUPDR]
BIC r1, r1, #0xC0
ORR r1, r1, #0x80
STR r1, [r0, #GPIO_PUPDR]
```

电路行为

- 未按下: PA3 被内部下拉拉到 0。
- 按下: PA3 连接到 +3V, 经按钮变成 1。
- 变化: 0 -> 1, 因此 rising edge。

易错点: pull-down 不是把引脚永久锁死为 0。外部强连接到 +3V 时, 输入仍会变成 1。下拉只是给“没人驱动时”一个默认低电平。

Slide 29: 配置 NVIC - 让 CPU 允许响应 EXTI3

这页在说什么

这页出现:

```
PUSH {LR,R0}
BL config_NVIC_in_C ; Enable Interrupt
POP {LR,R0}
```

它表达的是：在汇编主程序中调用一个 C 函数，去配置 NVIC，使能 EXTI3 中断。

为什么要 PUSH/POP?

BL config_NVIC_in_C 是函数调用。调用函数可能修改 R0 和 LR。如果主程序之后还需要原来的 R0/LR，就要先保存：

- PUSH {LR,R0}: 把 LR 和 R0 保存到栈。
- BL config_NVIC_in_C: 调用 C 函数配置 NVIC。
- POP {LR,R0}: 恢复 LR 和 R0。

NVIC 在链路中的位置

PA3 产生的是外部电平变化。EXTI3 负责检测这个变化。NVIC 负责决定 CPU 是否响应该中断。它们不是一个东西。

模块	中文	作用
GPIOA/PA3	输入引脚	读到外部电压变化
EXTI3	外部中断线	判断是否触发中断请求
NVIC	中断控制器	决定 CPU 是否进入 ISR
Vector Table	中断向量表	提供 ISR 入口地址

易错点： NVIC_EnableIRQ(EXTI3_IRQn) 不会自动配置 PA3，也不会配置上升沿。它只打开 NVIC 这一关。

Slide 30: 连接 PA3 到 EXTI3 - SYSCFG 配置

这页在说什么

这页是前面 Slide 7 的代码实现：让 EXTI3 的输入源选择 PA3。

代码中的一个值得警惕的地方

PPT 里的代码使用了 `RCC_AHB2ENR` 但 ORR 的宏名是 `RCC_APB2ENR_SYSCFGEN`。从命名上看，SYSCFG 通常属于 APB2 外设，因此概念上应当是开启 SYSCFG 的 APB2 时钟。课堂 PPT 重点不是纠结这处写法，而是理解：配置 SYSCFG 前也要打开 SYSCFG 模块时钟。

逻辑步骤

1. Enable SYSCFG clock: 让系统配置模块可用。
2. 读 SYSCFG_EXTICR1。
3. 清掉 EXTI3 的 source bits。
4. 写入选择 PA3 的编码。
5. 写回 SYSCFG_EXTICR1。

```
LDR r0, =SYSCFG_BASE
LDR r1, [r0, #SYSCFG_EXTICR0]
BIC r1, #SYSCFG_EXTICR1_EXTI3
ORR r1, #SYSCFG_EXTICR1_EXTI3_PA
STR r1, [r0, #SYSCFG_EXTICR0]
```

直白理解：

EXTI3 像一个插座，PA3/PB3/PC3 像几根可选的线。SYSCFG_EXTICR 的作用就是把 PA3 这根线插到 EXTI3 插座上。

Slides 31-32: Rising Trigger Selection - 让上升沿触发

这组页的核心

EXTI_RTSR1 是 Rising Trigger Selection Register。RT3 位控制 EXTI3 是否响应上升沿。

```
LDR r0, =EXTI_BASE
LDR r1, [r0, #EXTI_RTSR1]
ORR r1, #EXTI_RTSR1_RT3
STR r1, [r0, #EXTI_RTSR1]
```

为什么是 rising edge?

因为我们前面设置了 PA3 内部下拉：

- 未按下：PA3 = 0。
- 按下：PA3 接到 +3V，PA3 = 1。

所以按下瞬间是 0 -> 1，就是 **rising edge**（上升沿）。

如果配置错会怎样？

- 如果你只开 FTSR，不开 RTSR，那么按下按钮时可能不触发，松开时才触发。
- 如果 RTSR/FTSR 都不开，PA3 怎么变都不会触发 EXTI。
- 如果两者都开，按下和松开都触发。

易错点：上升沿/下降沿取决于电路和 pull-up/pull-down，不是死记“按键一定上升沿”。若采用上拉，按下可能是下降沿。

Slides 33-35: Interrupt Mask Register 和主程序死循环

IMR 是什么

EXTI_IMR1 是 Interrupt Mask Register。IM3 位控制 EXTI3 的中断请求是否被 mask。

- IM3 = 0: masked, 被屏蔽。EXTI3 即使触发, 也不能作为中断送往 NVIC。
- IM3 = 1: not masked, 未屏蔽。EXTI3 触发后可以送往 NVIC。

```
LDR r0, =EXTI_BASE
LDR r1, [r0, #EXTI_IMR1]
ORR r1, #EXTI_IMR1_IM3
STR r1, [r0, #EXTI_IMR1]
```

为什么主程序最后是死循环

嵌入式程序通常不会“退出”。电脑上的程序可以关闭, 但单片机程序通常上电后一直运行。因此主程序末尾常见:

```
stop
B stop
```

它的意思是永远跳回自己, 保持 CPU 不离开主程序。中断发生时, CPU 会暂时离开这个死循环去执行 ISR, ISR 结束后再回到死循环。

关键逻辑:

死循环不是“程序没用了”。它是主程序的常驻状态。外部事件通过中断临时打断它。

易错点: IMR 打开不等于 NVIC 打开。EXTI->IMR 和 NVIC_EnableIRQ 是两道门。

Slide 36: EXTI3_IRQHandler - 中断服务程序

这页在说什么

这页给出 ISR 框架。当 PA3 按下触发 EXTI3 后, CPU 执行 EXTI3_IRQHandler。ISR 里先检查 pending flag, 确认是不是 EXTI3, 然后执行任务, 最后清除 pending flag。

```
EXTI3_IRQHandler PROC
    LDR r0, =EXTI_BASE
    LDR r1, [r0, #EXTI_PR1]
    AND r1, #EXTI_PR1_PIF3
    CMP r1, #EXTI_PR1_PIF3
    BNE end
    ; Toggle LED
    ; Cleared flag by writing 1
    LDR r0, =EXTI_BASE
    LDR r1, [r0, #EXTI_PR1]
    ORR r1, #EXTI_PR1_PIF3
    STR r1, [r0, #EXTI_PR1]
end
ENDP
```

逐行解释

- 读取 EXTI_PR1: 看哪些 EXTI line pending。
- AND r1, ##EXTI_PR1_PIF3: 只保留 PIF3 这一位, 其他位不管。
- CMP r1, ##EXTI_PR1_PIF3: 判断 PIF3 是否真的为 1。
- BNE end: 如果不是 EXTI3 触发, 跳到 end。
- Toggle LED: 这里是实际业务逻辑, 可点亮或翻转 LED。
- 写 1 到 PIF3: 清除 EXTI3 pending flag。

为什么要清 pending flag

中断 pending flag 是“有事发生过”的记录。你处理完以后必须告诉硬件“我处理完了”。否则硬件认为这件事还在 pending, 可能 ISR 一结束又立刻进来。

非常重要: PR1 的 pending flag 通常是写 1 清除, 不是写 0 清除。初学者最容易反过来。

Stacking/unstacking 与这一页关系

按键触发后:

```
main dead loop
-> EXTI3 pending
-> hardware stacking
-> EXTI3_IRQHandler
-> clear pending flag
-> BX LR / exception return
-> hardware unstacking
-> return to main dead loop
```


Slides 37-38: 普通电机 vs 步进电机

为什么突然讲步进电机

本节后半部分开始从“输入中断”转向“输出控制”。按键/EXTI 可以作为输入事件，程序响应后可以控制 LED，也可以控制电机。步进电机是嵌入式里常见执行器。

Normal Motor 普通电机

普通直流电机通常两根线，给电流就转。它的问题是：转多少角度不精确。你只知道它在转，不天然知道它转了几度。

Stepper Motor 步进电机

步进电机通常多根线，例如 5-wire interface。它不是简单持续旋转，而是通过按顺序激活不同相位的电磁线圈，让转子一步一步移动。

项目	普通电机	步进电机
接口	通常 2 线	通常 4/5/6 线
控制方式	给电就转	按相位序列逐步激磁
位置控制	不精确，通常需要编码器反馈	每个脉冲对应固定角度，开环也可定位
典型用途	风扇、小车驱动	打印机、机械臂、指针、精密定位

一句话：普通电机像“油门”，步进电机像“走格子”。你发一个正确的相位变化，它走一步。

Slide 39: 28BYJ-48 与 ULN2803 驱动连接

这页在说什么

PPT 展示 STM32 的 GPIO 引脚 PB2、PB6、PB3、PB7 连接到 **ULN2803**（达林顿晶体管阵列驱动芯片），再连接到 28BYJ-48 步进电机的线圈。

为什么需要 ULN2803?

STM32 GPIO 不能直接驱动电机线圈。原因：

- GPIO 输出电流能力很弱。
- 电机线圈需要较大电流。
- 线圈断电时有反电动势，可能伤害芯片。

ULN2803 的作用是让 STM32 用小电流控制大电流，相当于电机驱动开关阵列。

控制链路：

```
STM32 GPIO output
-> ULN2803 input
-> ULN2803 output switches motor coil current
-> stepper motor phase changes
-> shaft rotates one step
```

四相控制

图中用四个输入控制四个线圈相位，通常可以抽象成 A、 $\bar{A}$ 、B、 $\bar{B}$ 四个控制位。程序输出一个 4-bit pattern，就决定哪几个线圈被激活。

易错点：GPIO 输出高/低不是直接“让电机转多少度”。它只是控制某个线圈是否通电。真正的转动来自连续的相位序列。

Slides 40-41: 步进电机内部结构与磁极激活

磁极控制的直白理解

步进电机内部有线圈。线圈通电后会产生磁场，变成电磁铁。转子上有永磁体。电磁铁的 N/S 极会吸引或排斥转子，让转子转到特定角度。

A 和 $\bar{A}$ 的含义

PPT 说：

- 如果 $A=1$ 且 $\bar{A}=0$ ，则 A 磁铁成为 South， $\bar{A}$ 磁铁成为 North。
- 如果 $A=0$ 且 $\bar{A}=1$ ，则 A 磁铁成为 North， $\bar{A}$ 磁铁成为 South。
- 如果 A 和 $\bar{A}$ 状态相同，磁铁不激活。

为什么相同状态不激活

这里可以先粗略理解为：A 和 $\bar{A}$ 是一对相反方向的控制端。只有一边开、一边关，线圈中才形成有效方向的电流；两边相同则没有有效激磁方向。

A	$\bar{A}$	结果
1	0	形成一个方向的磁场
0	1	形成相反方向的磁场
0	0	不激活
1	1	不按这种方式使用/不形成有效相位

B 和 $\bar{B}$

B 与 $\bar{B}$ 是另一组线圈相位，逻辑与 A/ $\bar{A}$ 类似。步进电机转动就是不断切换 A/ $\bar{A}$ /B/ $\bar{B}$ 的激活组合。

核心：步进电机不是靠“电流大小连续调速”完成定位，而是靠“磁场方向按顺序切换”吸引转子一步一步走。

Slide 42: Full-Stepping 全步进

什么是 Full-Stepping

Full-stepping（全步进）是一种步进电机控制方式。每次切换到下一个相位组合，转子走一个完整步距角。PPT 给出的顺时针序列是：

$$AB, \overline{AB}, \overline{AB}, AB$$

序列的意义

每个状态都激活两个线圈相位。随着相位组合变化，电磁场方向也跟着转动，转子被吸引到下一个稳定位置。

步骤	激活组合	直白解释
1	AB	A 相和 B 相共同吸引转子
2	$\bar{A}B$	磁场转到下一个方向
3	$\bar{A}\bar{B}$	再转一个方向
4	$A\bar{B}$	再转一个方向

如何反向转

反向转不是换硬件，而是把序列反过来：

$$A\bar{B}, \overline{AB}, \overline{AB}, AB$$

一句话：全步进每次走一个完整 step，力矩较大，因为通常两个相位同时激活。

Slides 43-44: Half-Stepping 半步进

什么是 Half-Stepping

Half-stepping（半步进）在全步进的相邻状态之间插入单相激活状态，所以每一步角度减半，分辨率变高，转动更细。

PPT 中的序列可理解为在双相和单相之间交替：

$$AB, B, \overline{AB}, \overline{A}, \overline{AB}, \overline{B}, A\overline{B}, A$$

具体符号方向依电机接线定义，考试/实验按 PPT 给出的相序写。

与 Full-stepping 对比

项目	Full-stepping	Half-stepping
每圈内部步数	少	多一倍
每次角度	一个完整 step angle	半个 step angle
转动平滑性	一般	更平滑
力矩	通常较大	某些单相状态力矩较小
控制序列长度	4 个状态	8 个状态

一句话：半步进不是让电机“半开”。它是在全步进相邻磁场方向之间增加中间状态，使角度分辨率翻倍。

易错点：半步进的步数翻倍，不代表速度自动翻倍。速度取决于你切换相位的频率。若每个状态延时不变，半步进完成一圈反而需要更多状态切换。

Slides 45-46: 一圈需要多少步

Rotor poles 与内部步数

PPT 说 rotor 有 8 pairs of poles, 不是 1 pair。意思是转子内部有多组磁极, 所以每个电气相位变化对应的机械角度更小。

Full-stepping 内部电机

全步进的相位序列有 4 个状态。转子有 8 对磁极, 所以内部电机每圈步数为:

$$4 \times 8 = 32 \text{ steps/revolution}$$

Half-stepping 内部电机

半步进序列有 8 个状态。转子有 8 对磁极, 所以内部电机每圈步数为:

$$8 \times 8 = 64 \text{ steps/revolution}$$

加上齿轮减速比

28BYJ-48 这种电机内部有齿轮减速。PPT 使用 gear ratio = 64。

所以输出轴一圈需要:

$$\text{Full step total} = 32 \times 64 = 2048 \text{ steps/revolution}$$

$$\text{Half step total} = 64 \times 64 = 4096 \text{ steps/revolution}$$

每一步角度

$$\begin{aligned}\text{Full step output angle} &= \frac{360^\circ}{2048} \approx 0.176^\circ \\ \text{Half step output angle} &= \frac{360^\circ}{4096} \approx 0.088^\circ\end{aligned}$$

考试记忆:

- Full-stepping: 内部 32 步/圈, 输出 2048 步/圈。
- Half-stepping: 内部 64 步/圈, 输出 4096 步/圈。
- 半步进分辨率更高, 但每圈需要更多控制状态。

Lecture 15 全链路总结

一、EXTI 外部中断配置完整流程

以 PA3 joystick up button 触发 EXTI3 为例：

1. 打开 GPIOA 时钟：RCC->AHB2ENR |= GPIOAEN。
2. 设置 PA3 为输入模式：MODER3 = 00。
3. 设置 PA3 内部下拉：PUPDR3 = 10。
4. 打开 SYSCFG 时钟。
5. 配置 SYSCFG_EXTICR1：选择 PA3 作为 EXTI3 source。
6. 配置 EXTI_RTSR1：启用 EXTI3 rising edge trigger。
7. 配置 EXTI_IMR1：解除 EXTI3 interrupt mask。
8. 配置 NVIC：enable EXTI3 interrupt。
9. 编写 EXTI3_IRQHandler：检查 pending flag，处理事件，清除 pending flag。

最常见错误：

- 忘记打开 GPIO 时钟。
- 忘记配置 SYSCFG，导致 EXTI3 没接到 PA3。
- 上升沿/下降沿选反。
- 只开 EXTI IMR，没开 NVIC。
- ISR 里忘记清 pending flag。

二、EXTI 与 NVIC 的区别

对象	属于哪里	作用
GPIO	外设	读取物理引脚电压
EXTI	外设/中断事件控制	检测边沿，产生 pending
NVIC	Cortex-M 内核	决定 CPU 是否响应中断
Vector Table	Flash/Code 区	保存 ISR 入口地址
ISR	程序代码	真正处理事件

三、步进电机控制完整模型

1. STM32 GPIO 输出 4-bit 相位控制信号。
2. ULN2803 放大/驱动电流。
3. 电机线圈按 A、 $\bar{A}$ 、B、 $\bar{B}$ 组合激活。
4. 磁场方向按顺序变化。
5. 转子一步一步对齐磁场。
6. 改变序列方向即可改变转向；改变切换频率即可改变转速。

四、关键词中英对照

英文	中文	解释
GPIO	通用输入输出	连接物理世界的数字引脚
EXTI	外部中断/事件控制器	把外部电平边沿转换成中断/事件请求
SYSCFG	系统配置模块	选择 EXTI line 来自哪个 GPIO port
RTSR	上升沿触发选择寄存器	允许 0->1 触发

FTSR	下降沿触发选择寄存器	允许 1->0 触发
IMR	中断屏蔽寄存器	决定 EXTI 请求能否送出
PR	挂起寄存器	记录中断是否已经发生
SWIER	软件中断/事件寄存器	用软件模拟触发 EXTI
NVIC	嵌套向量中断控制器	管理中断响应、优先级、嵌套
ISR	中断服务程序	中断发生后 CPU 执行的函数
Stepper Motor	步进电机	通过相位序列精准转动的电机
Full-stepping	全步进	每次走一个完整步距角
Half-stepping	半步进	每次走半个步距角，分辨率更高
Gear ratio	齿轮减速比	内部电机转动与输出轴转动的比例

最后一句:

Lecture 15 的本质不是背寄存器名字，而是建立两条硬件链路：**输入链路** PA3 -> EXTI3 -> NVIC -> ISR;
输出链路 GPIO -> ULN2803 -> 线圈相位 -> 步进电机转动。